

Sauti Yetu

System architecture and technical brief

Multilingual voice triage support for primary healthcare

Purpose. Help a health worker turn a patient's spoken complaint into a reviewable intake summary, urgency hint and routing suggestion, while supporting communication across languages.

The problem and the proposed interaction

Patients often describe symptoms in a mixture of Swahili and English. A conventional single-language intake flow can lose meaning at precisely the point where timing, symptoms and context need to be recorded clearly. Sauti Yetu begins with the patient's speech and presents structured information for the clinician to confirm.

The prototype combines speech recognition, language identification, deterministic knowledge-base reasoning and translation. A separate documentation agent can produce a report or flowchart from the consultation context. The clinician remains responsible for interpreting the output and deciding what to do next.

Patient experience	Clinician experience
Speak a complaint in a supported language or language mix.	Review the transcript, confirm suggested context and inspect the triage card.

Read or hear a translated response when the relevant services are available.	Inspect evidence links, enter measured vitals and generate consultation documentation.
--	--

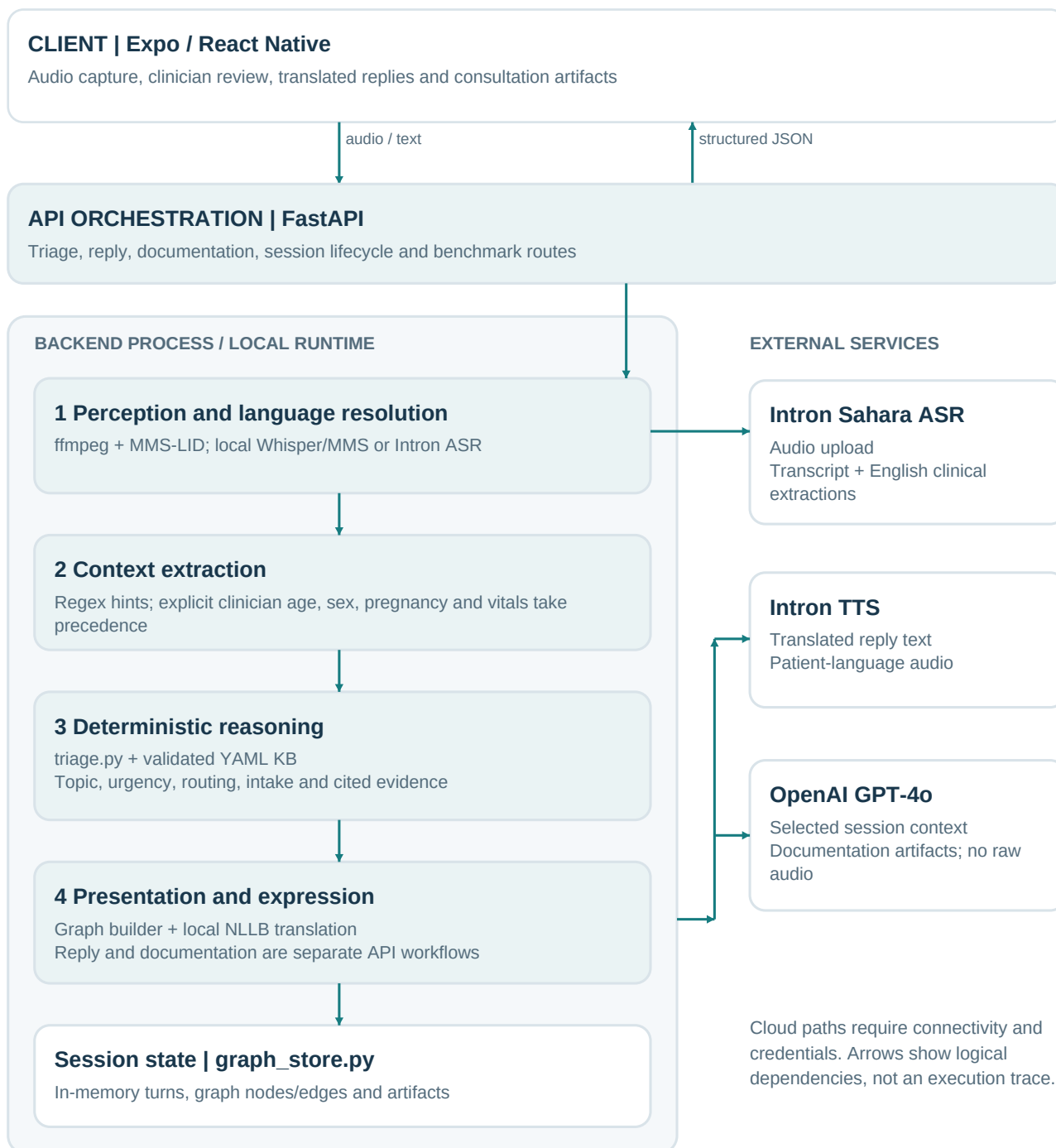
Scope of this document

Status: **working prototype, as described in the supplied brief.** This document explains the intended component responsibilities, data flows, API surface and operating constraints. It is based on the architecture Markdown, with evaluation caveats informed by the previously supplied JSON audit. Source code, deployment configuration and the referenced RESPONSIBLE_AI.md were not supplied for this review.

Decision support only. Outputs are suggestions for professional review, not diagnoses or autonomous clinical decisions. This document does not establish regulatory status or clinical validation.

Architecture at a glance

The client handles capture and review. FastAPI coordinates task-specific workflows. Local backend components perform normalization, extraction, deterministic reasoning and translation; selected workflows call external services.



Read the diagram: the central vertical path is a triage turn. Reply generation and documentation are separately requested workflows that reuse the session context. The client receives the resulting structured data through the API.

Deployment boundary: local models run on the backend host, which may be a clinic machine or a remote server. They are not necessarily running on the patient's phone. Offline use requires a reachable backend with model weights and dependencies already installed.

From audio to structured context

Client: capture, correction and review

The Expo / React Native client uses TypeScript and expo-router for a shared mobile and web interface. RecordSession and RecordControl capture speech; the typed API client packages platform-specific audio into multipart requests. The active consultation state links recording, the clinical card, the reasoning graph and generated artifacts.

ClinicalCard displays the intake summary and urgency hint; DifferentialPanel presents candidate conditions for review. LanguageSelect, LanguageChips and ReplySheet support language choice and playback. ProvenanceBadge and reasoning views expose the evidence behind the output. The interface should make suggested fields distinguishable from clinician-confirmed values.

Perception: normalization and transcription

The audio utilities use ffmpeg to prepare supported phone and browser formats as 16 kHz mono WAV. This creates a consistent input representation for the speech components. Models are lazy-loaded, so the first request can incur loading costs that are absent from later requests.

Component	How it works and what it produces
intron_client.py	Calls Sahara ASR with telehealth post-processing. The described response includes the patient-language transcript and English clinical extractions: summary, entities, candidate conditions and suggestions. Transport failures are normalized as IntronError.
asr_models.py	Hosts local Whisper and Meta MMS (mms-1b-all), plus audio preparation and MMS-LID. Local transcription provides an alternative speech path; equivalent cloud-style clinical extractions should not be assumed.
lid.py	Uses text-side language detection to refine an automatic selection after transcription. Audio language identification can guide the earlier stage. Neither a single language code nor closed-set LID fully describes every code-switch boundary.

Extraction: suggestions with explicit precedence

extract.py uses deterministic regular expressions to suggest age, sex and pregnancy context from the transcript or summary. Suggested values are marked auto_detected. Explicit clinician input takes precedence, preventing a speech-derived hint from silently replacing an entered value.

Vitals are not inferred from speech. The reasoning engine evaluates them only when a clinician supplies measured values. Negation, missing context and extraction errors still require review; deterministic extraction does not make its output automatically correct.

Example: a complaint containing "for 3 days" supplies duration evidence. A useful intake card preserves that evidence and leaves unmentioned fields unknown rather than completing them by assumption. This is an illustrative interaction, not a measured system result.

How the reasoning engine works

The triage layer is deterministic: given the same inputs, configuration and knowledge-base version, its rule evaluation should be reproducible. This makes the rule path inspectable. It does not remove uncertainty introduced by speech recognition, cloud extractions, incomplete complaints or the knowledge base itself.

Knowledge is maintained as reviewable data

knowledge/triage_kb.yaml defines topic mappings, routing departments, urgency rules, acuity discriminators, candidate-condition findings and age-banded vital-sign thresholds. knowledge_base.py loads and validates the structure on startup, allowing malformed configuration to fail early. Structural validation is separate from clinical review of the content.

Source metadata includes evidence_level, review_status, mapped_scale, ref and url. These fields let reviewers trace a rule to its reference and identify whether its mapping has been reviewed. The brief describes WHO IITT-derived discriminators; this document has not independently checked those mappings or their applicability.

Stage	Reasoning behavior
1. Collect evidence	Use the transcript, available English extractions and clinician-confirmed context. Preserve which component supplied each item.
2. Match and rank topics	Identify knowledge-base keyword matches, then rank topics by acuity and evidence weight. A topic links the complaint to a routing suggestion.
3. Derive urgency	Apply the configured discriminators and entered-vital thresholds. These adjustments may escalate the rule-derived urgency, rather than lower it.
4. Assemble the output	Return topic, urgency, department, intake fields, clarifying questions, candidate conditions, matched red flags and provenance.
5. Expose the reasoning	Build a graph showing how findings support the topic, urgency and red-flag output, then attach it to the consultation.

Candidate conditions and evidence

The brief describes a naive-Bayes-style representation of condition findings in the knowledge base. Its exact likelihoods, priors, missing-data handling and score calibration are not specified here. Candidate rankings should therefore be presented as reviewable suggestions, not diagnostic probabilities or confirmed conditions.

Important distinction. Escalate-only logic prevents a matched rule from lowering urgency within that stage. It cannot guarantee that all urgent evidence was recognized upstream. Likewise, a citation explains the rule used; it does not prove the individual output is correct.

GPT-4o is confined to the documentation workflow in the described architecture. The triage path still depends on model-generated transcripts and, in the cloud path, clinical extractions. Determinism applies to the reasoning engine, not to every upstream component.

Replies, documentation and memory

Translated replies: /api/respond

A health worker enters an English reply. translator.py uses local NLLB-200 distilled-600M to translate it into the patient's supported language; phrases.py provides pre-translated quick replies. Intron TTS then receives the reply text and produces speech for playback. The brief describes the audio being returned as base64 data.

Translation and speech synthesis are separate capabilities. Text translation can remain available locally when cloud TTS is unavailable. The described passthrough behavior for English, Pidgin and unknown language pairs means text may be returned unchanged; it should not be presented as a successful translation when no translation occurred.

Documentation: /api/command

A clinician supplies a typed command or a voice command that is first transcribed. agent.py sends selected consultation context to GPT-4o to generate a Markdown report or Mermaid flowchart, returned through the API as an artifact. Raw audio is not sent to the documentation model in the described design.

The prompt instructs the model to include a decision-support disclaimer, avoid introducing unrecorded symptoms or vitals, preserve EMERGENCY red flags and use the doctor's language. These are generation constraints, not guarantees. The clinician should review the artifact against the consultation record before using it.

Reasoning graph: evidence across turns

graph.py converts each triage result into a graph delta: new nodes and edges linking complaints, findings, topics, urgency and red flags. Provenance travels with these elements. graph_store.py deduplicates graph elements and keeps the turn history, giving the documentation workflow access to accumulated context instead of only the latest utterance.

Session behavior	Practical meaning
One session per consultation	A session identifier connects turns, graph updates and generated artifacts.
In-memory, process-local storage	State resides in one backend process. Restarting it clears the stored sessions.
Explicit end-session deletion	DELETE /api/session/{id} removes the session from the store.
No automatic TTL	Ending a conversation in the real world does not itself expire memory. The client must invoke deletion; otherwise the session can remain until process restart.
Multiple backend workers	Without shared state or appropriate routing, another worker may not have the requested session. Horizontal scaling needs a deliberate state strategy.

An in-memory store limits persistence, but is not by itself a complete retention policy. Logs, temporary files, client state and external-service handling are separate data locations that must be considered.

API surface and request lifecycle

Endpoint	Responsibility
GET /api/languages	Return the supported patient and doctor language lists.
GET /api/phrases	Return quick-reply phrases for the selected patient language.
POST /api/session	Start a consultation and establish its session context.
GET /api/session/{id}	Read the active consultation, including stored context and artifacts.
DELETE /api/session/{id}	Explicitly end and delete the stored consultation.
POST /api/triage	Accept audio and optional clinician context; coordinate perception, extraction, reasoning, translation and graph updates.
POST /api/respond	Translate a worker reply and request patient-language speech when TTS is available.
POST /api/command	Accept a clinician command and create a documentation artifact from session context.
POST /api/benchmark	Evaluate model transcripts against a supplied reference transcript.

A complete triage turn

- 1. Receive and prepare.** The client uploads the complaint and optional age, sex or measured vitals. The backend normalizes the audio and resolves the selected speech path. In automatic mode, audio LID can inform language selection.
- 2. Transcribe and extract.** The selected ASR produces text. Intron's telehealth path also returns English extractions. Text-side LID can then refine the detected language. Regex-derived context is merged with clinician input, preserving explicit values.
- 3. Reason and translate.** The deterministic engine applies the knowledge base and produces structured decision-support output. NLLB prepares doctor-language text where translation is supported.
- 4. Update and return.** The graph delta is appended to the session. The API returns the patient and doctor text, triage output and relevant consultation data for the client to render and review.

Keep workflow failures distinguishable

An ASR transport error, an unsupported translation pair, an unavailable documentation service and insufficient clinical evidence are different conditions. The API and UI should distinguish them so that a worker can correct the input, retry a service or continue with human assessment. The brief specifies normalized Intron errors; exact fallback triggers and response schemas require source-code verification.

Endpoint names and responsibilities above are taken from the supplied technical brief. They are an architecture reference, not a verified OpenAPI schema; request fields, response types, authentication and status-code behavior are not fully specified in the source.

Data handling and offline capability

Data boundary	Described behavior and qualification
Audio to Intron ASR	The cloud transcription path sends complaint audio. Audio itself may contain identifying information even if no separate identity field is supplied.
Text to Intron TTS	Speech synthesis receives reply text. The source's blanket "audio only" description is therefore specific to ASR, not all Intron traffic.
Context to GPT-4o	Documentation receives selected doctor-facing session context, not raw audio. De-identification is a stated design intention; the actual removal mechanism is not documented in the brief.
Temporary audio	The brief states that temporary WAV files are deleted after processing. Cleanup on error and interruption paths requires implementation verification.
Session data	Stored in process memory with explicit deletion and no TTL. Restart clears this store; that does not establish deletion from logs or external services.

What can operate without external APIs

Capability	Offline operating condition
Local transcription	Whisper or MMS weights, runtime and audio tools must already be installed on the backend; the client must still reach that host.
Context extraction and reasoning	Regex extraction and the YAML knowledge base need no API key. Their effectiveness depends on the available transcript and extraction inputs.
Text translation and quick replies	NLLB weights and supported language mappings must be available locally; cached phrases can support known replies.
Spoken reply	The described Intron TTS path needs connectivity and credentials. No local speech-synthesis replacement is specified.
Generated documentation	The GPT-4o documentation path needs connectivity and credentials. No local documentation agent is specified.

Capability is different from demonstrated reliability

Removing API keys is not the same test as disabling network access. A credible offline demonstration uses preinstalled models, an explicitly isolated runtime and the same test clips, then records successful requests, failures and feature availability. The fallback path must also account for the absence of Intron's English clinical extractions.

The earlier 64-clip JSON export recorded `offline=false` and `noise=false` and contained no separate offline or controlled-noise outputs. The architecture may support those evaluation modes, but that export does not demonstrate their completion. No new offline or noise measurements are asserted here.

How to assess and maintain the system

Evaluate each layer and the full workflow

Evaluation layer	Measurement and interpretation
Speech recognition	WER and CER against independently verified human references. Report model versions, language settings, errors and clip counts.
Code switching	Switch-point WER around annotated transitions. Document token indexing, window size and coverage of actual language boundaries.
Runtime	Latency, real-time factor and failure rate. Separate model loading from warm inference; identify hardware and network dependencies.
Downstream support	Topic, urgency, routing, entity and slot results with separate annotation denominators. Inspect urgency confusion and missed reference red flags.
Noise and offline behavior	Matched clean/noisy clips across defined SNR levels; a distinct network-disabled local run. Report quality changes and unavailable features.

The reported implementation map

Area	Primary modules / technology
Client	app/; Expo, React Native, TypeScript, expo-router; typed API and consultation state.
API and speech	backend/app.py; FastAPI, Uvicorn; intron_client.py, asr_models.py, lid.py; ffmpeg.
Extraction and reasoning	extract.py, triage.py, knowledge_base.py, knowledge/triage_kb.yaml.
Replies and documentation	translator.py (NLLB-200), phrases.py, tts_client.py, agent.py (GPT-4o).
Session and evaluation	graph.py, graph_store.py, benchmark.py; generate_primary_report.py and generate_benchmark_report.py; jiwer, matplotlib, fpdf2.

Priorities beyond the prototype

Verify model-selection and fallback behavior; audit references and knowledge-base mappings; make unsupported languages and failed services visible; validate documentation against session facts; and define an explicit session lifecycle. Before a broader deployment, document authentication, access controls, transport protection, logging behavior and worker/session routing, which are not specified in the supplied brief.

Source and evidence note

Primary source: "Sauti Yetu - System Architecture & Technical Brief", supplied Markdown attachment, sections 1-7. Evaluation caveat: previously supplied Pasted text.txt benchmark export, generated 15 September 2026. Module names, technologies and intended behavior are attributed to the brief. Explanations of dependencies and operating implications are editorial analysis. No repository inspection, live endpoint testing, clinical-source verification or regulatory assessment was performed.